

数据库系统原理

一. 数据:

1. 什么是数据: 依据视角而定

- 现实生活 — 可识别的抽象符号, 即描述事物的符号记录
- 计算机 — 所有能被计算机处理的符号的总称
- 数据库 — 存储的基本对象

2. 种类: 数字, 文本, 时间, 向量, 图片, 音频

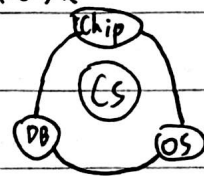
3. 类型: (1) 结构化数据: 关系数据(表)

(2) 半结构化数据: 键值对 Key-Value, Markdown, XML, JSON, 图, 向量

(3) 非结构化数据: 文本文档, 电子邮件, 图像, 音频, 视频

4. 关系数据: 二维表关系

5. 大数据: 5V理论



6. 生成式大模型: LLM — 大语言模型; LVM — 视觉大模型;

MLM — 多模态大模型; LGM — 图大模型.

7. Big Data, DB in AI, 大模型训练数据, 多源、异构、多模态大数据, 数据处理流程

8. 数据库(Database, DB): 一组有关联的数据集合; 长期存储在计算机中, 有组织、管理共享

基本特征: 按一定的数据模型组织、描述和存储; CRUD; 支持并发查询处理

9. 数据库系统(DS): 数据库管理系统(DBMS)

传统文件存储的问题

方便应用开发

管理 DB 的级

MySQL
PostgreSQL
ORACLE
Oracle

Declarative, MySQL, PostgreSQL

Microsoft SQL server

二. 课程概述:

1. 关系表: 以大学为例

物理层
逻辑层
视图层

2. 数据库, 数据库管理系统

3. 数据视图: 数据抽象 — 视图层、逻辑层、物理层

schema 结构

4. 模式与实例, 数据模型、关系模型、数据库模式

instance 当前状态实例

5. 数据库语言: 定义语言 DDL; 操纵语言 DML, 同时属于 SQL.

6. 数据库设计. E (Entity, 实体) - R (Relationship, 关系) 模型

7. 数据库引擎、事务管理器

8. 数据库和应用体系结构

9. 数据库用户和管理员 DBA: 管理员 职责

10. 数据库系统的历史

11. 前沿发展方向

- ① 模式定义
- ② 存储结构存取方法
- ③ 模式及物理组织修改
- ④ 数据访问授权
- ⑤ 日常维护

Chapter 2 关系模型

一. 关系数据库的结构:

1. 关系: 给定 $D_1, \dots, D_n$, 关系 r 是 $D_1 \times \dots \times D_n$ 的一个子集, 元组 $(a_1, \dots, a_n)$ 满足 $a_i \in D_i$. 记录

2. 属性: 每个关系由一组属性组成。属性的域: 取值范围 变动, 随时变化

属性的值: 要求是原子的, 不能是多值/复合属性 空值: 值缺/不存在

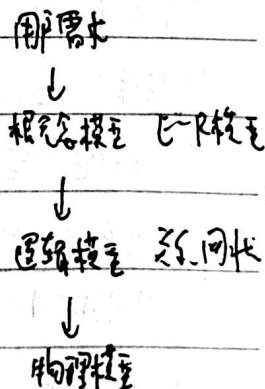
3. 关系模式: $R = (A_1, \dots, A_n)$, $A_1, \dots, A_n$ 为属性, 则 $r(R)$ 是 R 的一个关系模式. 表的定义, 稳定

4. 关系实例: 特指某时刻存储在数据库中的实际元组集合 动态, 随时变化

二. 数据库模式:

1. 定义: 包含多个关系的数据库

2. 注意不把所有数据放入同一个关系, 防止数据冗余和数据稀疏



三. 码/键:

1. 超码: 让 K 为 R 的一个子集, 如果 K 能匹配 $r(R)$ 里的一个独特元组

2. 候选码: 无多余属性, 最小超码 可以多属性

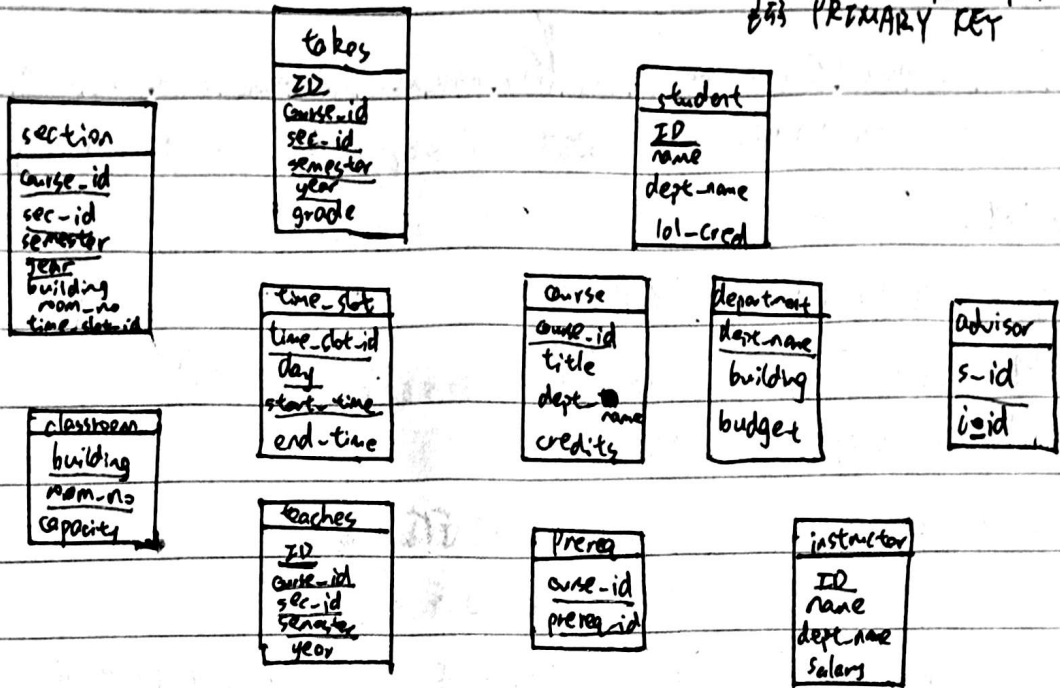
3. 主码: 唯一确定的编号

4. 外码: 刻画表和表的关系 参照完整性的约束

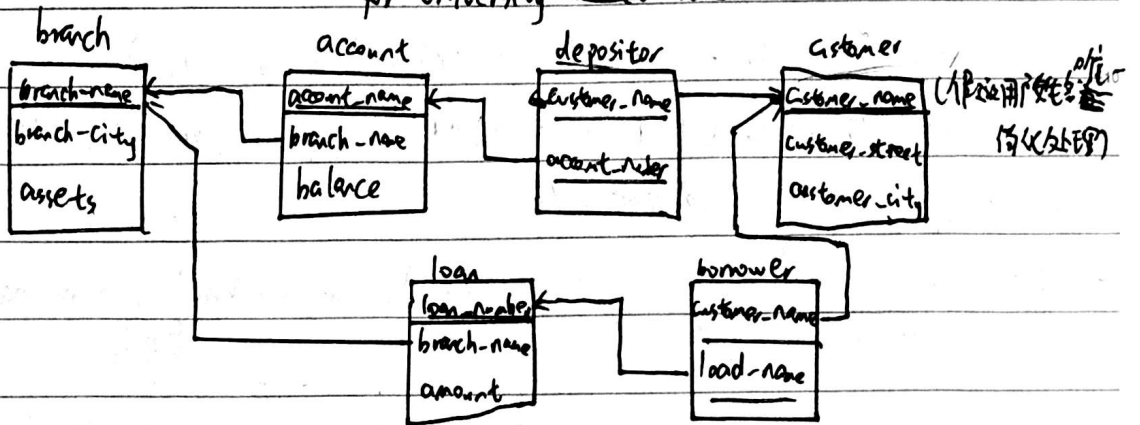
四. 模式图

五. 关系查询语言: 过程式 - 关系代数 核心: 集合运算
非过程式 - SQL (域语言)

MYSQL PRIMARY KEY
 主键 PRIMARY KEY



Schema for University Database



Schema for Bank

- 关系代数: 输入关系, 输出关系
- 一元运算: ① 选择 $\sigma_p(r) = \{t | t \text{ 满足条件 } p\}$
 predict: 谓词: 选择条件 $\wedge$ and $\neg$ not
 $\vee$ or $= t < r >$
- ② 投影 $\pi_{A_1, \dots, A_k}(r)$ 选 $A_1, \dots, A_k$ 字段. 集合属性值 (投影可能丢失主码)
- ③ 重命名 $\rho_x(E)$ 由一般对结果表表达式重命名. E (expression) 重命名为 x . eg: $\rho_d(\text{account})$
- 二元运算: ① 集合并 $r \cup s = \{t | t \in r \text{ or } t \in s\}$ r, s 必须同元, 且相容
- ② 集合差 $r - s = \{t | t \in r \text{ and } t \notin s\}$
- ③ 笛卡尔积 $r \times s = \{tq | tq \in r \text{ and } tq \in s\}$
 (可拆元)
- 广义投影: $A_1 \dots A_k$ 字段
 $\rightarrow F, F$ 运算
 保留主码 - 一定存在

eg: $\pi_{customer_name}(\sigma_{branch_name='Jia\ Dong'}(\sigma_{borrow_name \neq 'computer' \wedge borrow_name \neq 'borrower' \wedge borrow_name \neq 'borrower' \wedge borrow_name \neq 'borrower'}(borrower \times loan)))$

关系表达式: 附加RA操作: θ ; $r \cap s = r - (r - s) = \{t | t \in r \text{ and } t \in s\}$

② $r \bowtie s$: 自然连接, 相同属性相连并接

eg: $R = (A, B, C, D) \quad S = (B, D, E)$

① $r \bowtie s = \pi_{r.A, r.B, r.C, r.D, s.E}(\sigma_{r.B=s.B \wedge r.D=s.D}(r \times s))$

② 当 $R \cap S = \emptyset$ 时 $r \bowtie s = r \times s$

c.f. θ -join: $r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$ 与自然连接无关

③ 外连接 保留未相连的连接 c.f. 内连接, 不连接

① 左外连接 ② 右外连接 ③ 全外连接 没的值 NULL 即可.

留左 留右 留全

④ 除: $r \div s = \pi_{R-S}(r) - \pi_{R-S}((\pi_{R-S}(r) \times s) - \pi_{R-S, S}(r))$

s 所有有在, 里面找得到 $r \div s = \{t | t \in \pi_{R-S}(r) \wedge \forall u \in s (t \div u)\}$

⑤ 赋值运算: $\leftarrow$ 提高可读性 $\leftarrow$

⑥ 聚合操作: $\pi_{A_1, \dots, A_n}(\sigma_{A_1, \dots, A_n}(f(A_1, \dots, A_n)))$ $f(A_1) = \text{sum(balance)} \text{ as } (\text{sum_balance})$

Chapter 3 SQL 介绍 数据非, 外码依赖, distinct

组成 { 数据定义语言 DDL
数据操纵语言 DML

SELECT ...
FROM ...
(where).

primary key 主码 非空且唯一 *

自然连接

仅指定, 默认

join ... using (...) 或 Natural Join 宽松

附加基本运算: 更名运算 $(old_name) \text{ as } (new_name)$

构造自身 select distinct T.name

from instructor as T, instructor as S

where T.salary > S.salary and S.dept_name = 'Biology'

字符串 ... where building like ' % Watson % '

元组的顺序: order by desc 降序

where 子句谓词 $(a, b) = (a', b')$ between: 范围

集合运算: $\cup, \cap, -$

空值: NULL

聚合函数: avg() 平均 count() 总数 distinct: 去重

group by 分组

Having 子句 where: 在 group 之前 having: 在 group 之后

嵌套子查询: select-from-where 可以接表 类似“循环”, 效率偏低

eg 1: 集合包含 in: 在...里面 (not in)

(= some) 与 in 一样

Some 子句 $S = \text{some}(\frac{a}{b}) \checkmark$ $S \neq \text{some}(\frac{a}{b}) \checkmark$

(#some is not in 不一样)

all 子句

(#all is not in 一样)

空关系测试 exists (not exists)

(= all) 与 in 不一样

谓词逻辑等价: $X - Y = \emptyset \Leftrightarrow X \subseteq Y$

检查元组测试 unique ≤ 1 not unique? 2

视图 (View): create view 分发权限; 只能为数据 虚拟的

creat view v as <query expression>

导出表:

① With 子句:

标量子查询:

数据库的修改: ~~delete~~ delete from ... where ...

Case 语句

case

when ... then

when ... then

else

end

insert

updates

Chapter 4 中级SQL (Chapter 5 高级SQL)

关系实例, 自然连接 (natural join)

连接类型: 左/右/全外连

连接条件: ① natural join - (所有相同)

join ... using (↓)

② ~~using~~ on 连接

full-outer join

外连接不要把 on 条件移到 where,

③ on 更严格

视图: 物化视图

数据提交 回滚

select [distinct] 输出结果

事务 (transaction): commit / rollback

from 关系1 join 关系2 on 连接条件

完整性约束:

where 行筛选

域约束:

group by 分组属性 < having 组筛选条件

非空约束:

order by 排序属性

Unique 约束:

check 约束:

参照完整性约束:

数据库修改: 插入时检查完整性条件

外码为空认为满足条件

断言 create a view or check 每次都要检查

create table student

(ID varchar(5)

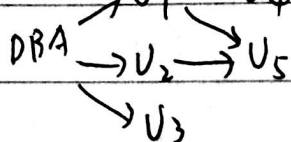
name varchar(20) not null

score numeric(3,0), default 0,

primary key (ID)

(check (score >= 0))

权限、授权:



grant [权限]

on [表]

to user-list, public

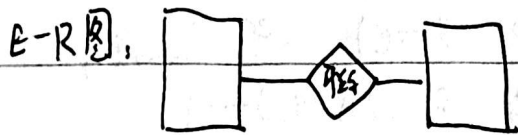
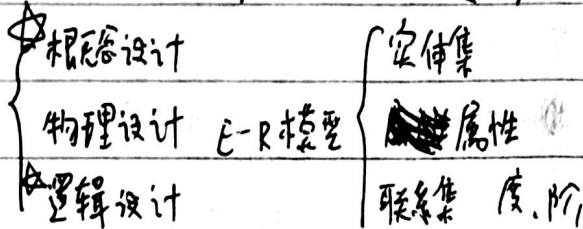
收回权限: revoke 把 权限给相应的角色 from

加密: AES, DES

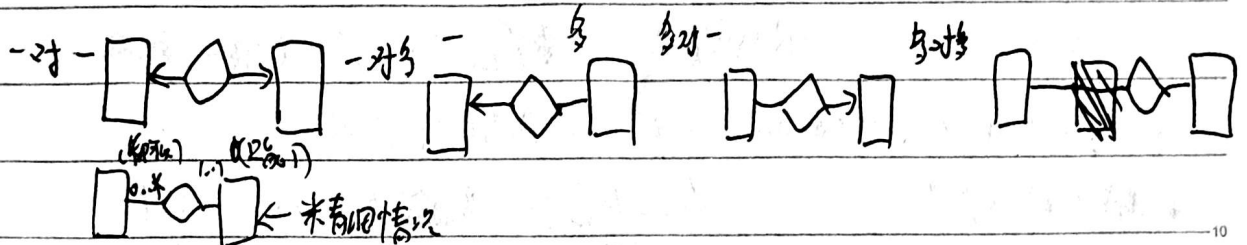
Chapter 6 数据库设计与E-R模型

现实世界 $\rightarrow$ ER chapter 7: 优化

设计过程概览: 设计数据库模式; 设计访问更新数据的程序; 设计控制数据访问安全机制



(一对一多箭头)



二元 $\rightarrow$ 二元, 加入属性及实体集

弱实体集: 依赖于某实体集. 例如大学db里家长与学生

特化: 自上而下的设计 $\downarrow$ ISA: is a

泛化: 自下而上的设计 $\uparrow$

E-R图 $\rightarrow$ 关系模式 注意: 一对多对一不用为每个关系新建table

Chapter 7 数据库优化 (函数依赖)

好的关系: 大关系模式信息重复出现, 更新需同时更新 小关系模式: 信息丢失
冗余不一致 冗余不一致

$\alpha \rightarrow \beta$: α 唯一确定 β . eg: $\begin{bmatrix} A \\ B \\ C \end{bmatrix} \rightarrow \begin{bmatrix} D \\ E \end{bmatrix}$ $A \rightarrow B \times B \rightarrow A \checkmark$

超码 $K \rightarrow R$ 传递 $\Rightarrow (K \rightarrow R) \wedge (\exists a c K \wedge a \rightarrow R)$

完全依赖: 最小化情况

无损连接

闭包: $\beta \subseteq \alpha$: $\alpha \rightarrow \beta$; $\alpha \rightarrow \beta \vee \alpha \rightarrow \gamma \vee \alpha \rightarrow \beta \wedge \beta \rightarrow \gamma$ $\alpha \rightarrow \gamma$
(五步法) 合并: $\alpha \rightarrow \beta, \alpha \rightarrow \gamma$, 则 $\alpha \rightarrow \beta\gamma$
分解: $\alpha \rightarrow \beta, \alpha \rightarrow \gamma$ 且 $\alpha \rightarrow \gamma$
传递: $\alpha \rightarrow \beta$ 且 $\beta \rightarrow \gamma$, 则 $\alpha \rightarrow \gamma$

算法：先用自反、增广律、传递律，直到 F^+ 不变。NP-hard 函数依赖

属性集的闭包：能推导出的所有属性 $\alpha \rightarrow \beta$ 在 F^+ 内 $\iff \beta \subseteq \alpha^+$

$\emptyset \rightarrow \alpha^+ = U \iff$ 超码 检查子集是否等于 U ，若均不能 $\implies$ 又是候选码

应用：检查超码、候选码；检查冗余；求 F 的闭包

正则覆盖：函数依赖存在冗余，求函数依赖的最小化，令对 F_c 无冗余属性
在左右部无冗余属性

eg: $R = (A, B, C)$ $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ $F_c = ?$

$B \rightarrow C$ 则 $AB \rightarrow C$ 里 A 冗余； $A \rightarrow C$ 里 C 冗余。 $F_c = \{A \rightarrow B, B \rightarrow C\}$

计算候选码：①分主 $L/R/N/LR$ ②若 $X_{LN}^+ = U$ ，则是唯一候选码

eg: $F = \{A \rightarrow C, C \rightarrow D, D \rightarrow A\} = F_c$ $U = \{A, B, C, D\}$ $R \subseteq U, F$

1) 对 F_c $X_{LN}^{UN} = X_L = B$ $Y_{LR} = ACD$ $X_{LN}^+ = \{B\} \neq U$

2) 检查 BA, BC, BD ，发现均可 $\implies$ 三者均为候选码 不用考虑 Y_R ，因为仅右部无冗余属性

eg: $R \subseteq U, F$ $V = \{0, 1, 2, 3, 4\}$ $F = \{S \rightarrow D, D \rightarrow S, Z \rightarrow R, R \rightarrow Z, R \rightarrow O, O \rightarrow R\}$

1) $F_c = F$ 2) $X_{LN} = \emptyset$ $Y_{LR} = S D I B O$ 3) $Q^+ = \{O\}$

4) 2个不行，发现候选码 R 可以是 $(QSD), (QDO), (QSR), (QDR), (QSZ), (QDZ)$

X_{LN} ：仅在左部无 Y_{LR} ：在右部出现 先 X_{LN} ，然后挑 Y_{LR} 里冗余由少到多加

无损连接分解 eg: $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$ 候选码种类数可能不同。
 $R_1(A, B, C)$ $R_2(C, D)$ $R_3(D, E)$

表格法：

	A	B	C	D	E
$R_1(A, B, C)$	a_1	a_2	a_3	$b_4^{a_4}$	$b_5^{a_5}$
$R_2(C, D)$			a_3	a_4	$b_5^{a_5}$
$R_3(D, E)$				a_4	a_5

分解目标： $(U \dots F_n)^+ = F^+$

范式 1NF：无多值 原子性要求

2NF：1NF + 无部分依赖 每个非码属性完全依赖于候选码属性

3NF：2NF + 无传递依赖 不存在传递依赖（非主属性对候选码的传递依赖）

4NF：所有决定属性是候选码 一定无损分解且保持

(5分)

eg: $R < U, F >$, $U = \{A, B, C, D, E\}$, $F = \{A \rightarrow B, C \rightarrow D, B \rightarrow C, C \rightarrow D, B \rightarrow E\}$.

1) $F_c = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$.

2) 候选码: $X_{LR} = A$ $Y_{LR} = BC$ ($A^+ \neq U$).

$(AB)^+ = U$; $(AC)^+ = U$. AB, AC 为候选码

PCIMF

3) 弱码属性: D, E ; D 由 C 决定, 是 AC 的部分依赖 $\therefore$ 不满足 2NF, 2NF

3) $U_1 = \{A, B, C\}$ $F_1 = \{A \rightarrow B, B \rightarrow C\}$ 若有候选码就不用再加一条边

$U_2 = \{B, C, E\}$ $F_2 = \{B \rightarrow C\}$

$U_3 = \{C, D\}$ $F_3 = \{C \rightarrow D\}$

eg: $R < U, F >$, $U = \{A, B, C, D\}$, $F = \{A \rightarrow C, C \rightarrow A, B \rightarrow AC, D \rightarrow AC, B \rightarrow A\}$.

1) $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow A\}$

2) 候选码: $X_{LR} = BD$ $Y_{LR} = AC$

$(BD)^+ = U$. 则 BD 为唯一候选码. A 部分依赖于 BD (仅 $B \rightarrow A$)

2NF

3) $U_1 = \{A, C\}$ $F_1 = \{A \rightarrow C, C \rightarrow A\}$

$U_2 = \{A, B\}$ $F_2 = \{A \rightarrow B, B \rightarrow A\}$

$U_3 = \{A, D\}$ $F_3 = \{A \rightarrow D, D \rightarrow A\}$

每条 F_c 均可拆成

4) $U_4 = \{B, D\}$, $F_4 = \emptyset$

$\Rightarrow$ 候选码保持.

要加有候选码

$R_{i+1} = \langle \text{候选}, \emptyset \rangle$.

表格

4) A B C D

R_1 a_1 b_{12} a_3 b_{14}

R_2 a_1 a_2 b_{23} b_{24}

R_3 a_1 b_{32} b_{33} a_4

R_4 a_1 a_2 a_3 a_4 无投

☆ 每一条函数依赖都可以在分解后的关系里找到
因此必是候选码保持

由后推向前, 作提高作: Cache $\rightarrow$ 内存 (SSD) $\rightarrow$ 磁盘 (磁带或硬盘) $\rightarrow$ 磁带/光盘
稳定性 非易失性, 速度, 成本, 可靠性

B 树: 中间节点指向指针 Chapter 12-13 物理存储系统, 数据库存储结构 (偏序为主)

Chapter 14 索引

搜索码, 索引页, 索引文件, 区分索引代, 索引值, 范围查询, 主/二级索引

主索引 Primary index 主索引: 与文件顺序一致
散列索引

此外非顺序索引: 什么是散列, 什么是

一. 顺序索引: 适合范围查找 按搜索码有序

主索引: 记录的数据文件按某搜索码指定的顺序排序 索引顺序文件

辅助索引/非聚集索引: 和文件排列顺序不一致

索引: 树形结构, 叶节点存放数据

1) 稠密索引: 为每个搜索码都构建索引项, 每个关键字按搜索码不是主码时仍可能一致

稀疏索引: 只能找到一个范围只记录数据

多级索引: 内索引:

外索引: 一般是稀疏索引, 为了省大小

均以文件块形式组织

稠密 vs 稀疏 (选地址位置两局都有)

索引维护: 删除

非 hit 世界不碰书

索引插入: 稀疏若有空可直落插 (一般二级文件块用来出数据)

非聚集索引

每个索引项指向一个 bucket (可能放多个)

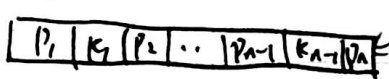


(因为分散存, 所以二级索引一般是稠密索引)

每个几乎都需要读一次文件块, 所以很慢

二. B+ 树索引和 B 树索引: eg: 10, 12, 27, 5, 9, 15, 30, 7, 17, 26, 每个节点 ≤ 4 个指针

溢出块



n: 指针数量

B+ 树是 平衡树

树节点

中间节点 $\lceil \frac{n}{2} \rceil \sim n$ 个孩子

叶子节点 $\lceil \frac{n-1}{2} \rceil \sim n-1$ 个搜索码

非叶子 ≥ 2 孩子

Pi: 指针 K: 搜索码 (已排序, $K_1 < \dots < K_{n-1}$)

非叶子节点: 多级稀疏索引

(K_{i-1}, K_i)

物理上取不相邻

平衡树: 适合字 K, $H \approx \lceil \log_{n/2} (K) \rceil$ 层数 叶子 (0 ~ n-1) 个搜索码

B+ 树插入: 分裂

(所有关键字存在叶子里 一定会出现一次)

删除合并

B- 树: $P_1, K_1, P_2, \dots, P_n, K_n, P_{n+1}$

缺点: 节点数少, 不一定可存底层两页

按散列桶组织

缺点: ① 难以扩容 ② 存储空间变大 ③ 维护成本高

三. 散列索引: 适合找有特定值的记录

① 有主码表取值 (哈希) 溢出桶 动态索引

② 随机 Hash: 二级索引

四. 多码访问: account 表网络文件 位图索引

五. 索引键: create (UNIQUE) index (index name) on (relation name)

Chapter 15 查询处理

步骤: 语法分析 & 翻译 $\rightarrow$ 优化 $\rightarrow$ 执行

SQL ~~非代数~~ $\Rightarrow$ 关系代数 过程式

eg: select salary
 from instructor
 $\text{where salary} < 75000$
 $\Rightarrow \pi_{\text{salary}} (\sigma_{\text{salary} < 75000} (\text{instructor}))$ 先选择 $\vee$ 先投影
 σ 先行 π 后列

CPU 开/关, I/O, 网络等因素 主导: 磁盘访问

eg: 块传输次数 t_T t_S $b \cdot t_T + s \cdot t_S$, buffer 没存现代, 若这是最好情况
 ≈ 0.1 ≈ 4

一. 选择操作: 文件扫描, 索引扫描

- A1: 线性搜索 (前提: 数据文件块顺序存放) $\frac{br}{2}$ 各种情况均可用
- A2: 主索引 - 候选码 B⁺树 $(h_i + 1) \cdot (t_T + t_S)$ 块非顺序存放每块都要扫描
- A3: 主索引 - 非码属性 $h_i \cdot (t_T + t_S) + t_S + t_T \cdot b$ $\leftarrow$ 包含记录数
- A4: 二级索引 (主码属性: $(h_i + 1)(t_T + t_S)$ 非码属性: $(h_i + 1)(t_T + t_S)$)
- A5: 主索引 - 比较 $6A_{2,4}(r)$ 扫描: $6A_{5,6}(r)$ 可以不用索引
- A6: 二级索引 - 比较 每块都要 I/O; 性能不如 A1
- A7-9: 合并 $6A_{7,9}(r)$
- A10: 析取/取反

★ 二. 排序操作: 给出整个归并排序的过程
 外排非序 - 归并 可放入内存快速排序即可

M: buffer 的数据块大小 N 归并 (设 $N \ll M$)

创建 M 个归并段 I/O 若 $N \geq M$ 时多趟归并 每块 1 个 output, $M-1$ 个用作归并段
 归并 设有 br 个 block, 初始归并段 $\lceil \frac{br}{N} \rceil$, 总归并趟数 $\lceil \log_{N/M} \lceil \frac{br}{N} \rceil \rceil$
 写 R_i I/O 字数 $br (2 \lceil \log_{N/M} \lceil \frac{br}{N} \rceil \rceil + 1)$ b_b : 每个归并段的 block 数
 $2br (\text{read} + \text{write})$ 文件块数 $\uparrow$ 块数 $\frac{br}{b_b}$
 (最后一次可不写) $12 \times (2 \lceil \log_2 \frac{12}{5} \rceil + 1) = 60$

寻址时间 $2 \lceil \frac{br}{N} \rceil$ 总 seek 次数: $2 \lceil \frac{br}{N} \rceil + \lceil \frac{br}{b_b} \rceil (2 \lceil \log_{N/M} \lceil \frac{br}{N} \rceil \rceil - 1)$
 eg: $2 \times \lceil \frac{12}{3} \rceil + \lceil \frac{12}{7} \rceil (2 \lceil \log_{2/5} \lceil \frac{12}{3} \rceil \rceil - 1) = 2 \times 4 + 12 \times (2 \times 2 - 1) = 44$ 次

三. 连接操作:

嵌套循环连接, 以 r_{mos} 为例
 $\begin{matrix} \uparrow & \uparrow \\ \text{外层} & \text{内层} \end{matrix}$

若每块内存放 n block, 需 $b_{r1} + n \cdot b_2$ 全表扫描

块嵌套循环连接

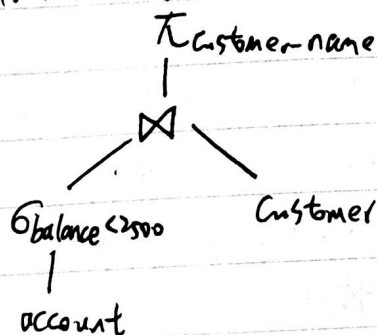
索引嵌套连接 让元组少的表作为外层关系

* 归并连接, 散列连接

* 四. 其他复杂连接:

五. 表达式 ~~连接~~ 执行:

1. 物化: 每个放到硬盘, 可适用于任何计算



双缓冲可优化物化, 2个buffer. 当其中一个满后停止往硬盘写, 另一个启用. 将中间查询结果全部算出来并存在临时表, 再给下一个运算符用.

2. 流水线: 不一定适用于所有情况
 一个运算符产出一行数据, 立即传给下一个运算符处理.

需求驱动 自上而下
 生产驱动 自下而上 元组-buffer

☆☆☆ eg. 假设每个数据块只能容纳一个元组, 内存可保留3个数据块, 给出针对下列元组各个属性的归并排序过程.

$t_1=(j, 17)$ $t_2=(u, 2)$ $t_3=(e, 1)$ $t_4=(w, 13)$ $t_5=(p, 3)$ $t_6=(m, 8)$

$t_7=(v, 4)$ $t_8=(z, 11)$ $t_9=(n, 6)$ $t_{10}=(i, 9)$ $t_{11}=(h, 2)$ $t_{12}=(b, 12)$

创建归并段 $r_{11} = \{t_1^e, t_4^j, t_2^u\}$

$r_{21} = \{t_3, t_{11}, t_5, t_2, t_4\}$

(k=2为3) $r_{12} = \{t_6^m, t_5^p, t_9^w\}$

$r_{22} = \{t_{12}, t_{11}, t_{10}, t_9, t_7, t_8\}$

按字典序

$r_{13} = \{t_8^n, t_7^v, t_9^z\}$

↓

$r_{14} = \{t_{12}^b, t_{11}^h, t_{10}^i\}$

写回硬盘

$r_{31} = \{t_{12}, t_3, t_{11}, t_{10}, t_1, t_6, t_9, t_5, t_2, t_7, t_{11}, t_8\}$

Chapter 16 查询优化

优化查询树 Sort 把选择移到关系里去
有逻辑即可

一. 关系表达式的转换, 如查询树

- 步骤:
1. 产生逻辑上与给定表达式等价的表达式
 2. 对产生的表达式以不同方式进行标注, 产生不同的查询执行计划
 3. 估计每个执行计划的代价, 选择估计代价最小的执行计划

等价规则: 1. $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

2. $\sigma_{\theta}(\sigma_{\theta_1}(E)) = \sigma_{\theta_1}(\sigma_{\theta}(E))$

3. $\pi_{t_1}(\pi_{t_2}(\dots(\pi_{t_n}(E))\dots)) = \pi_{t_n}(E)$ (最后都要 t_1)

选择与投影交换 4. $\sigma_{\theta}(E_1 \times E_2) = E_1 \times \sigma_{\theta} E_2$

$\sigma_{\theta_1}(E_1 \times \sigma_{\theta_2} E_2) = E_1 \times \sigma_{\theta_1 \wedge \theta_2} E_2$

投影交换 5. $E_1 \times \sigma_{\theta} E_2 = \sigma_{\theta} (E_1 \times E_2)$

6. $(E_1 \times E_2) \times E_3 = E_1 \times (E_2 \times E_3)$

连接顺序优化 7. $(E_1 \times \sigma_{\theta_1} E_2) \times \sigma_{\theta_2 \wedge \theta_3} E_3 = E_1 \times \sigma_{\theta_1 \wedge \theta_2} (E_2 \times \sigma_{\theta_3} E_3)$, 要求 θ_2 只涉及 E_2 属性

8. 当 θ_1 所有属性只涉及一个表达式, $\sigma_{\theta_1}(E_1 \times E_2) = (\sigma_{\theta_1}(E_1)) \times E_2$

当 θ_1, θ_2 分别只涉及 E_1, E_2 时, $\sigma_{\theta_1 \wedge \theta_2}(E_1 \times E_2) = (\sigma_{\theta_1}(E_1)) \times (\sigma_{\theta_2}(E_2))$

9. θ 只涉及 E_1, E_2 属性 $\pi_{t_1 \cup t_2}(E_1 \times \sigma_{\theta} E_2) = (\pi_{t_1}(E_1)) \times (\pi_{t_2}(E_2))$

10. $E_1 \cup E_2 = E_2 \cup E_1; E_1 \cap E_2 = E_2 \cap E_1$

11. $(E_1 \cup E_2) \cap E_3 = (E_1 \cap E_3) \cup (E_2 \cap E_3)$

12. $\pi_{t_1}(E_1 \cup E_2) = (\pi_{t_1}(E_1)) \cup (\pi_{t_1}(E_2))$

二. 表达式执行代价估计:

* 简单选择: $b_A = a(r)$ ~~n_r~~ $\frac{n_r}{|R(r)|} \pi$ 元组

$$b_{A \leq v}(r) = \begin{cases} 0 & v < \min(A, r) \\ n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)} & \text{else} \\ n_r & v \geq \max(A, r) \end{cases}$$

复杂选择: 合取: $n_r \cdot \frac{s_1 \dots s_n}{n_r^n}$

析取: $n_r (1 - (1 - \frac{s_1}{n_r}) \dots (1 - \frac{s_n}{n_r}))$

取反 $b_{\neg \theta}(r): n_r - \text{size}(b_{\theta}(r))$

* 估计值

三. 执行计划选择: 启发式/动态规划

- ① 更早地进行关系选择
- ② 更早投影选择
- ③ 约束严格的选择

Chapter 17 事务(transaction)

一组原子性的操作单元

核心 { 并发控制
恢复

ACID { A: 原子性
C: 一致性
I: 隔离性
D: 持久性

eg: 1. read(A)

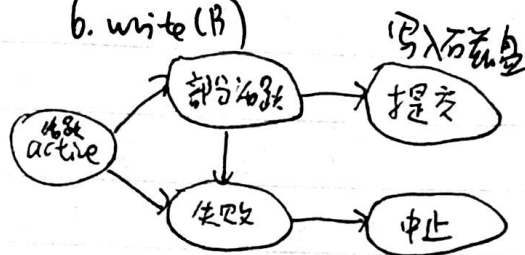
2. A := A - 50

3. write(A)

4. read(B)

5. B := B + 50

6. write(B)



事务的状态: 活跃

并发执行: 充分利用资源; 降低~~阻塞~~时间

二. 调度: 事务执行的先后顺序 串行: 先T₁再T₂ | 先T₂再T₁

非串行: T₁, T₂ 交织在一起 2个 A' + B' = A + B 即可

可串行化调度能保证状态一致性 不能 read 和 write 后写覆盖前写

{ 冲突可串行性
视图可串行性

(1) 冲突可串行性 仅 read(read) + read(read) 不产生, rtw/wtr/utw 均冲突不能交换

冲突等价: 交换顺序可以从 S 变到 S', 则 S 与 S' 等价, 若转化后得到的是串行顺序, 则

S 是冲突可串行化调度。

* (2) 视图可串行性 2个 schedule 结果一致 (最后一次 W 相同)

三. 可恢复调度:

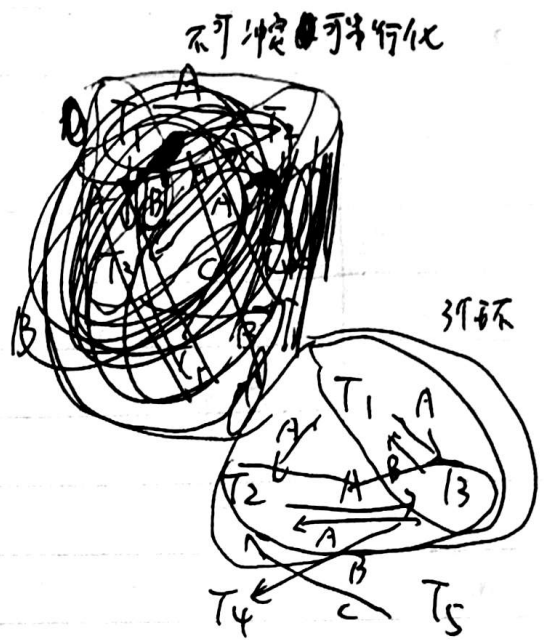
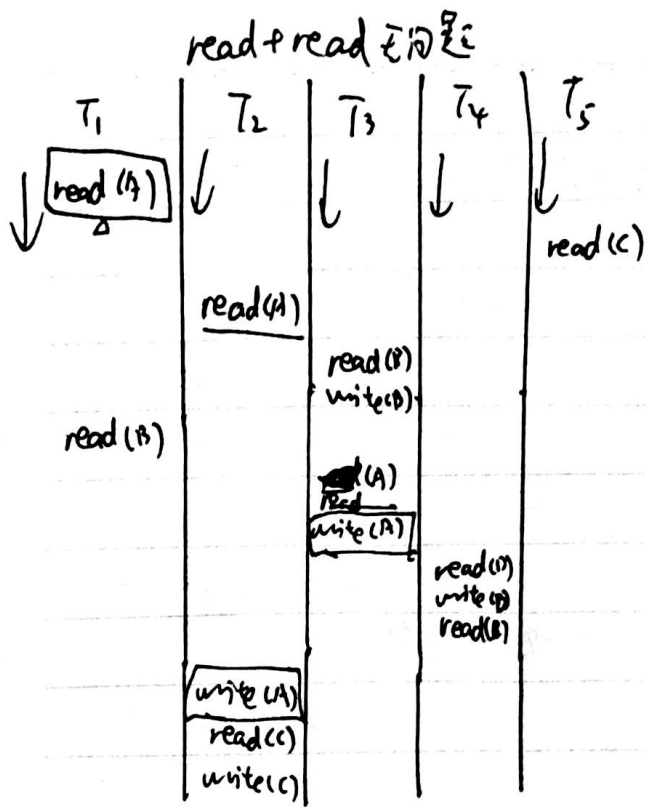
级联回滚: 确保无级联回滚调度

SQL 中的事务定义:

四. 可串行性检测: 优先图

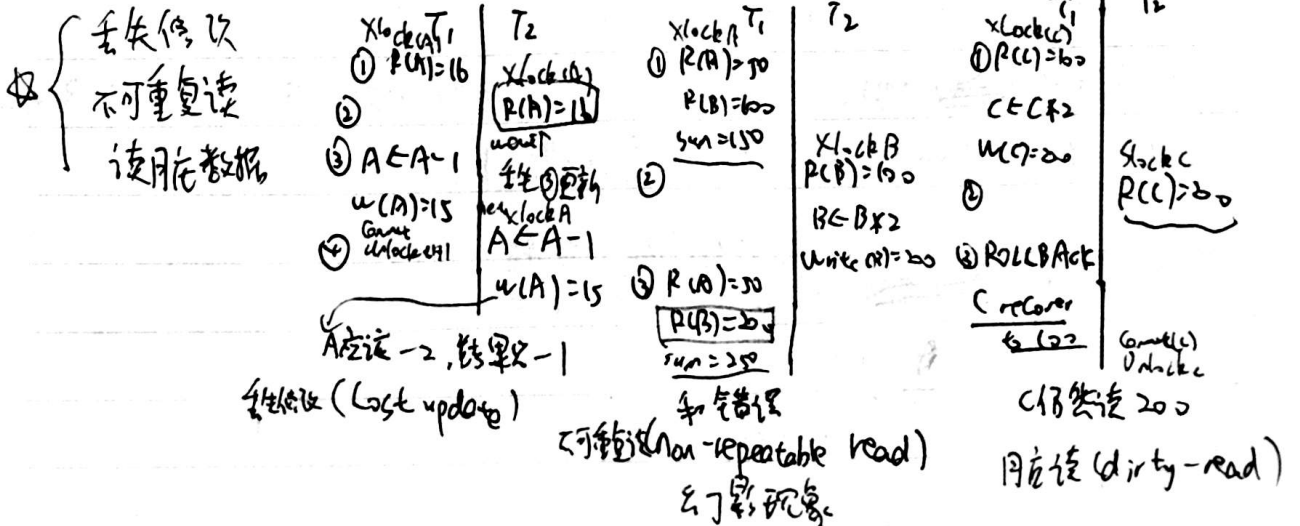
并发执行与可串行性检测

冲突可串行化 $\Leftrightarrow$ 优先图无环, 进行拓扑排序
可得到顺序



Chapter 18 并发控制 解决 I, 隔离性 C: 一致性

一. 并发控制中的问题: Read(X): 读 X Write(X): 写 X



二. 基于锁的协议: 加锁, 锁住

排他性: X 锁 读写
共享性: S 锁 只读

锁请求专用: 并发控制管理器

$xlockA$ $unlockA$

平滑两阶段锁: 所有排他锁都要升级到

强两阶段锁: 所有锁都要升级到 $commit$

锁相容矩阵

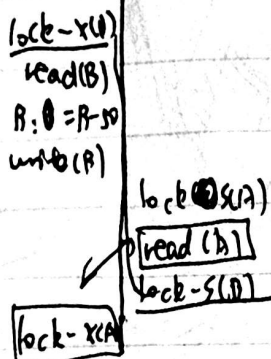
	S	X
S	true	false
X	false	false

两阶段锁协议: 保持冲突可并行化, 但不能死锁
 一. 增长阶段: 只加锁不解锁
 二. 收缩阶段: 只解锁不加锁
 关键点: 最后获得加锁的位置
 锁升级: $lockS \rightarrow lockX$
 锁降级: $lockX \rightarrow lockS$

锁的自动获取

锁的实现, 锁管理器 数据结构: 锁表, 哈希表

死锁: T_3 | T_4 相互等待, 死锁
两阶段(包括平滑/32) 都不能避免死锁协议
 $X \quad X+S$



饥饿: 一个 transaction 等待加锁, 同时有数据共享锁。
要加排他锁的只能长期等待。
($X-lock$)

三. 基于图的协议

偏序 $\rightarrow$ D: 数据库 graph 较简单的图协议, 树协议只允许 X 排他锁
好处: 避免死锁, 可以更早释放高优先级锁被 T 锁 $\rightarrow$ 被 T 锁

缺点: 容易回滚, 要加锁范围变大(因为有序)

基于时间戳的协议 可以避免死锁 按照时间戳排序 判断最新写入时间
 $TS(t_i) \leq W-timestamp(Q)$ 已经被覆盖, rollback; $>$ 没回滚更新 $R-timestamp$
 $\rightarrow \max(R-timestamp, TS(t_i))$

四. 死锁处理: 检测死锁 $\leftarrow$ 图时间戳直接检测
两阶段: 检测死锁

抢占 - 老的强利高的释放
非抢占 - 老的等待年轻的释放, 年轻的回滚

基于超时的机制: 检测饥饿, 让长时间的回滚。

死锁检测: 等待图 $G=(V, E)$

如果有环, 让某个 transaction 回滚 $\leftarrow$ 全rollback
检测死锁 检测回滚 $\rightarrow$ 检测死锁
都令 rollback: 让死锁解除即可(但难检测)

* 五. 粒度度

Chapter 19 恢复系统 解法 A: 原子性 D. 持久性

在正常执行过程中保留信息用于故障恢复
恢复数据库到一致性状态

易失性存储器: 内存 - 缓冲区 $\xleftarrow{\text{input(buffer)}}$
非易失性: 硬盘 - 物理块 $\xleftarrow{\text{output}}$
稳定存储器: 备份, 稳定备份

每个事务 T_i 有其私有工作区 B_x 直接写回硬盘

恢复与原子性: 显式页(创建备份), 基于日志 log-based recovery
copy 信息有放回

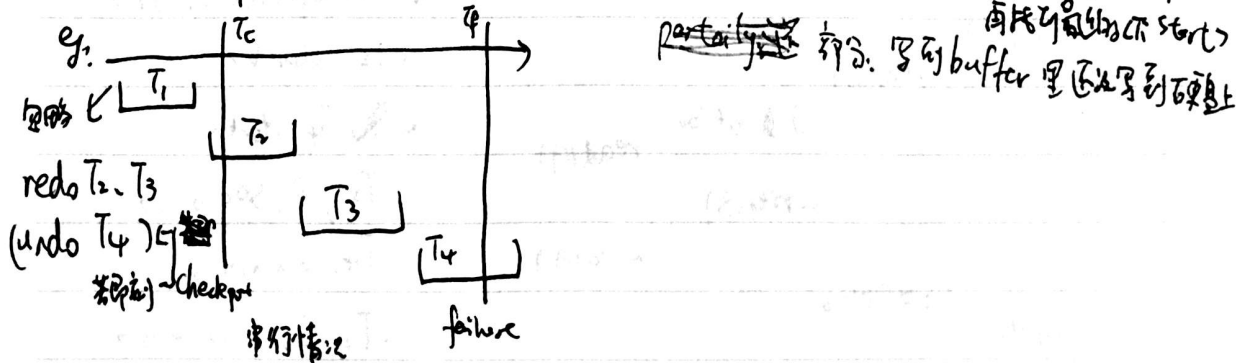
一. 基于日志的恢复:

当事务 T_i 开始时, 记录 $\langle T_i \text{ start} \rangle$ 日志. $\langle T_i, X, V_1, V_2 \rangle$ 修改 $\langle T_i, \text{commit} \rangle$ 提交
把 X 值由 V_1 改为 V_2

两类方式:
延迟数据库修改 - 通过读取日志来修改数据 好处: 已知所有操作日志, 能前滚也能回滚
即刻数据库修改 - 一边写日志一边修改数据 可 redo (重写)

undo(T_i): 取消当前事务中已经进行部分的操作 redo: 已写完的日志

检查点(checkpoint): 定期把 buffer 内容回收到硬盘上去, 每次前滚由检查点扫描到 checkpoint



二. 恢复算法: 针对并发情况 $\langle \text{checkpoint list} \rangle$

从日志 failure 处向前扫描直到 checkpoint L 记录停止

若 $\langle T_i \text{ commit} \rangle$, 把 T_i 加到 redo 已提交但未写进数据库, 故需要 redo.
若 $\langle T_i \text{ start} \rangle$ 但无 $\langle T_i \text{ commit} \rangle$, 加到 undo 未提交但更新已写入数据库, 需要 undo.
对检查点 $\langle \text{checkpoint L} \rangle$ 里的各记录, 若没有 undo, 加到 redo

三. 缓冲区管理

缓冲池在内存里 当 commit/checkpoint 时写回硬盘 (提高 I/O 效率 ^{减少 I/O 次数})
 任何操作日志先行再进行操作 WAL Write-ahead logging (多用于分布式场景)
 写回数据块时为 1-1 冗余
 数据库 buffer 主要在内存, 有时创建在机内存, 但会有双写问题 (多 I/O 次数)
 非写必性有错需 故障率低, 解决方案通常双备份 (确保时间尽量同步)
 (看硬盘)

eg: 下图中包含 4 个并发事务 T_1, T_2, T_3, T_4 , 其中数据 A, B, C 的初始值为: $A=800, B=500, C=300$. 若日志采用 "immediate database modification", 且在事务 T_2 开始时创建 1-checkpoint, 给出该调度的日志序列以及 A, B, C 的最终结果. 若在事务 T_2 提交前发生故障, 应如何恢复并解释原因.

T_1	T_2	T_3	T_4	
				< T_1 start >
read(A)				< T_3 start >
		read(B)		< $T_1, A, 800, 900$ >
$A := A + 100$				< checkpoint { T_1, T_3 } >
write(A)				< T_2 start >
	read(C)			< T_4 start >
		$B := B + 100$		< $T_3, B, 500, 600$ >
		write(B)		< T_4 commit >
		read(A)		< $T_2, C, 300, 400$ >
	$C := C + 100$			< T_2 commit >
read(A)	write(C)			< $T_1, B, 600, 1500$ >
read(B)				< T_4 commit >
$B := B + A$				< $T_3, C, 400, 100$ >
write(B)				< T_3 commit >
		read(C)		
		$C := 100$		
		write(C)		

- ① 看到 commit T_4 redo 已提交
- ② T_2 仅 start undo 未提交
- ③ T_1, T_3 undo